Implementation Plan: N8N Hands-On Intermediate Assessment

Topic 1: Production Workflow Planning & Design Patterns

Document Navigation

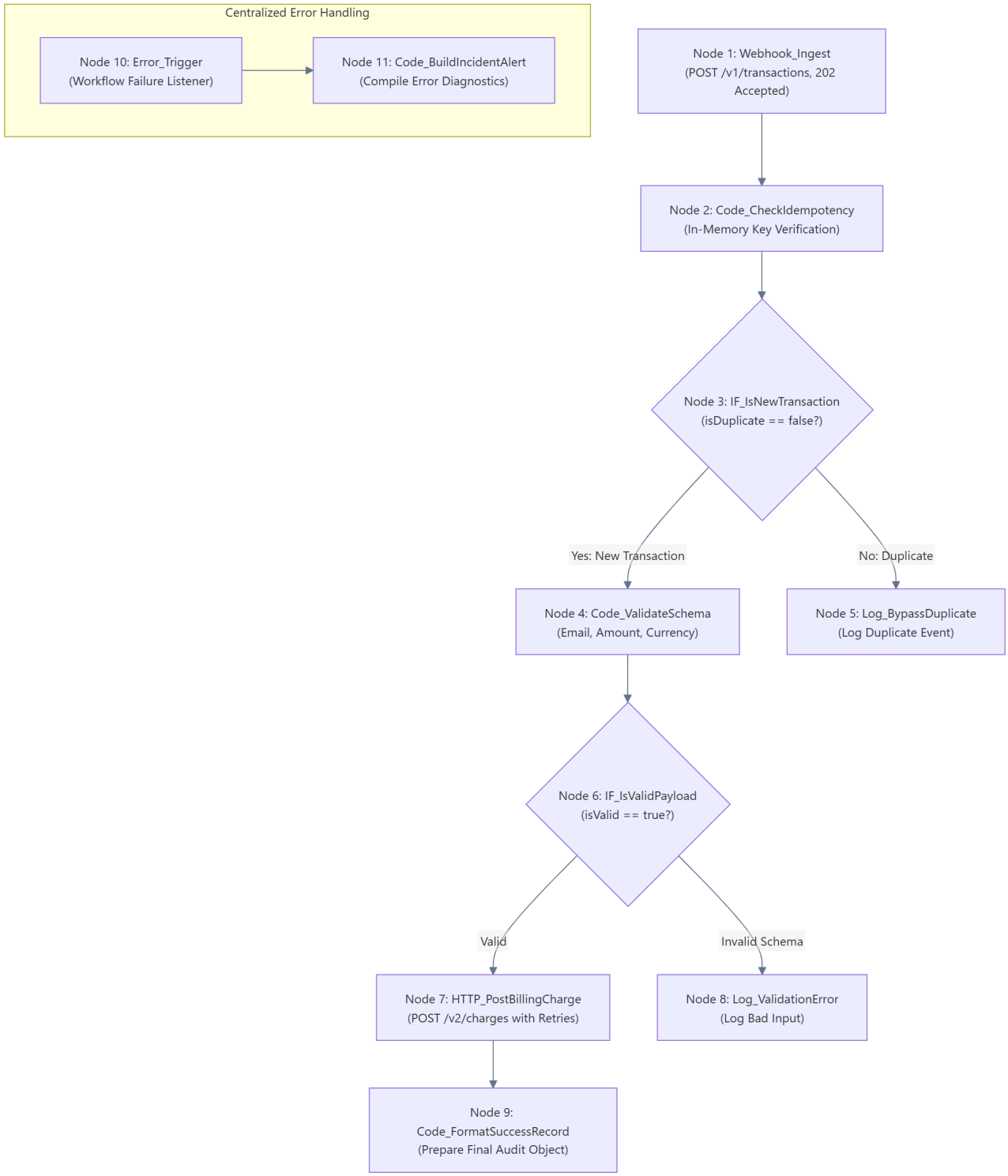
Concept Guide | Requirement Document | **Step-by-Step Implementation Plan** | Test Plan

1. Objective

Build an idempotent, production-ready transaction processing workflow in n8n that ingests customer payment requests via webhook, validates payload schemas, prevents duplicate executions, calls an external billing API with retry policies, and routes fatal failures to an incident monitoring handler. A zero-blocker in-n8n mock server is included so learners can execute and test all paths without external API dependencies.

2. Canvas Architecture Overview

All rights reserved. Tayana Academy • Confidential Document



3. Prerequisites & Environment Setup

- [ ] n8n instance version 1.0.0 or higher.
- [ ] Postman, Thunder Client, or cURL on the local machine.

All rights reserved. Tayana Academy • Confidential Document

- [ ] Optional: Configure `.env` for production execution logging:

```
# Enable database persistence for executions
DB_TYPE=postgresdb
DB_POSTGRESDB_HOST=localhost
DB_POSTGRESDB_PORT=5432
DB_POSTGRESDB_DATABASE=n8n_prod
DB_POSTGRESDB_USER=n8n_admin
DB_POSTGRESDB_PASSWORD=SecurePassword123!

# Prune execution history to prevent disk exhaustion
EXECUTIONS_DATA_PRUNE=true
EXECUTIONS_DATA_MAX_AGE=168
EXECUTIONS_DATA_PRUNE_TIMEOUT=3600
```

---

4. Step-by-Step Practical Build Guide

Step 1: Mock Billing API Setup (Choose One of 3 Options)

To test the external billing charge step without requiring a paid merchant gateway (Stripe/Authorize.net), select one of the following mock options:

Option A: Free Cloud Mock API via DummyJSON Custom Response (Recommended - Zero Install)

DummyJSON Custom Response generates an instant, persistent cloud mock REST API returning customized JSON:

1. Navigate to **https://dummyjson.com/custom-response** in your browser.
2. In the **JSON Response** editor, paste the mock charge response payload:

```
{
  "chargeId": "ch_dummyjson_882910",
  "status": "succeeded",
  "amount": 249.99,
  "currency": "USD",
  "timestamp": "2026-10-15T12:00:00.000Z"
}
```

3. Set **HTTP Status Code** to `201 Created` (or `503 Service Unavailable` when testing failure retry backoff).
4. Click **Generate API URL**.
5. Copy the generated live endpoint URL (e.g. `https://dummyjson.com/c/your-unique-id`).
6. In n8n, paste this URL into `HTTP_PostBillingCharge` as the target URL.

Option B: Postman Mock Server (Cloud Mock Endpoint)

If you use Postman for API testing and team collaboration:

1. Open **Postman** and create a new Collection named `n8n Intermediate Assessment`.

All rights reserved. Tayana Academy • Confidential Document

2. Add a new Request:
   - **Method**: `POST`
   - **Path / Name**: `/v2/charges`
3. In the top right of the request pane, click **...** $\rightarrow$ **Add Example**.
4. In the Example configuration:
   - Set Status Code to `201 Created`.
   - Set Body (JSON):

```
{
  "chargeId": "ch_postman_mock_4421",
  "status": "succeeded",
  "amount": 249.99,
  "currency": "USD"
}
```

5. Save the Example.
6. In the left sidebar, click `...` on the `n8n Intermediate Assessment` collection $\rightarrow$ select **Mock Collection**.
7. Name the mock server `n8n Billing Mock` and click **Create Mock Server**.
8. Copy the generated **Postman Mock URL** (e.g. `https://{{mock-id}}.mock.pstmn.io/v2/charges`).
9. In n8n, paste this Postman mock URL into `HTTP_PostBillingCharge`.

Option C: In-n8n Local Webhook Mock Server (Offline Sandbox)

If running in an isolated or offline local environment without internet access:

1. In n8n, create a new workflow named `Mock Billing Server`.
2. Add a **Webhook Node**:
   - **Node Name**: `Mock_Billing_Webhook`
   - **HTTP Method**: `POST`
   - **Path**: `mock/v2/charges`
   - **Response Mode**: `Using 'Respond to Webhook' Node`
3. Connect an **IF Node** named `IF_SimulateOutage`:
   - Value 1: `={{ $json.body.amount }}`
   - Operation: `Equal`
   - Value 2: `999`
   - **True Branch**: Connect to **Respond to Webhook Node**:
     - **Response Code**: `503`
     - **Response Body**: `{"error": "Billing Gateway Temporarily Unavailable", "code": "GATEWAY_TIMEOUT"}`
   - **False Branch**: Connect to **Respond to Webhook Node**:
     - **Response Code**: `201`

All rights reserved. Tayana Academy • Confidential Document

- **Response Body (Expression)**:

```
{{
  {
    "chargeId": "ch_mock_" + Math.random().toString(36).substring(2, 10),
    "status": "succeeded",
    "amount": $json.body.amount,
    "currency": $json.body.currency,
    "timestamp": new Date().toISOString()
  }
}}
```

4. Save and activate the workflow. *(Recommendation: Use Option A via DummyJSON Custom Response or Option B via Postman for instant, zero-blocker testing without port configuration).*

Step 2: Configure Main Ingestion Webhook (`Webhook_Ingest`)

1. Create a new workflow named `Topic 1 - Production Workflow Planning`.
2. Add a **Webhook Node**:
   - **Node Name**: `Webhook_Ingest`
   - **HTTP Method**: `POST`
   - **Path**: `v1/transactions`
   - **Response Mode**: `Immediately`
   - **Response Code**: `202`
   - **Response Data**:

```
{
  "message": "Transaction accepted for processing",
  "status": "ACCEPTED"
}
```

> **Important: ** Webhook Endpoint URLs by Environment:****
>
> **Webhook Endpoint URLs by Environment:**
>
> - **Production URL (Active Workflow)**: `https://n8n.tayanasolutions.com/webhook/v1/transactions` - **Test / Staging URL (Canvas Debugging)**: `https://n8n.tayanasolutions.com/webhook-test/v1/transactions` *(Use this when clicking 'Listen for test event' in the canvas)* - **Local / Docker Sandbox (Fallback)**: `http://localhost:5678/webhook/v1/transactions` - **Environment Dynamic Configuration**: `{{ $env.N8N_WEBHOOK_BASE_URL || 'https://n8n.tayanasolutions.com/webhook' }}/v1/transactions`

All rights reserved. Tayana Academy • Confidential Document

Step 3: Implement Idempotency Verification (`Code_CheckIdempotency`)

Add a **Code Node** to verify transaction uniqueness against an in-memory execution cache:

- **Node Name**: `Code_CheckIdempotency`
- **Mode**: `Run Once for Each Item`
- **Language**: `JavaScript`

```
// Global memory cache simulation inside n8n workflow static data
const staticData = $getWorkflowStaticData('global');
staticData.processedTransactions = staticData.processedTransactions || [];

const body = $input.item.json.body || $input.item.json;
const transactionId = String(body.transactionId || '').trim();

if (!transactionId) {
  return {
    json: {
      isDuplicate: false,
      hasId: false,
      payload: body,
      errorMsg: 'Missing required transactionId'
    }
  };
}

// Check if transaction ID already exists in cache
const isDuplicate = staticData.processedTransactions.includes(transactionId);

if (!isDuplicate) {
  // Store key with limit to prevent memory bloat
  staticData.processedTransactions.push(transactionId);
  if (staticData.processedTransactions.length > 500) {
    staticData.processedTransactions.shift(); // Evict oldest
  }
}

return {
  json: {
    transactionId: transactionId,
    isDuplicate: isDuplicate,
    hasId: true,
    payload: body
  }
};
```

Step 4: Add Duplicate Routing (`IF_IsNewTransaction`)

Add an **IF Node**:

- **Node Name**: `IF_IsNewTransaction`
- **Conditions**:
  - Value 1: `={{ $json.isDuplicate }}`
  - Operation: `Equal`

All rights reserved. Tayana Academy • Confidential Document

- Value 2: `{{ false }}`

Connect the **False Branch** to a **Code Node** named `Log_BypassDuplicate`:

```
return {
  json: {
    event: "TRANSACTION_BYPASSED",
    reason: "Duplicate transactionId detected",
    transactionId: $json.transactionId,
    timestamp: new Date().toISOString()
  }
};
```

Step 5: Implement Schema Validation (`Code_ValidateSchema`)

Connect the **True Branch** of `IF_IsNewTransaction` to a **Code Node**:

- **Node Name**: `Code_ValidateSchema`
- **Mode**: `Run Once for Each Item`
- **Language**: `JavaScript`

```
const payload = $json.payload || {};
const errors = [];

// 1. Email validation using standard RFC 5322 regex
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!payload.customerEmail || !emailRegex.test(payload.customerEmail)) {
  errors.push("Invalid or missing 'customerEmail'");
}

// 2. Amount validation (must be positive numerical value)
const amount = Number(payload.amount);
if (isNaN(amount) || amount <= 0) {
  errors.push("Field 'amount' must be a positive number greater than 0");
}

// 3. Currency validation (ISO 4217: 3 uppercase letters)
const currencyRegex = /^[A-Z]{3}$/;
if (!payload.currency || !currencyRegex.test(payload.currency)) {
  errors.push("Field 'currency' must be a valid 3-letter uppercase ISO code (e.g. USD, EUR, GBP)");
}

const isValid = errors.length === 0;

return {
  json: {
    transactionId: $json.transactionId,
    isValid: isValid,
    validationErrors: errors,
    sanitizedPayload: {
      transactionId: $json.transactionId,
      customerEmail: String(payload.customerEmail || '').toLowerCase().trim(),
      amount: amount,
      currency: String(payload.currency || '').toUpperCase().trim(),
      description: payload.description || 'Standard Order Charge'
    }
```

All rights reserved. Tayana Academy • Confidential Document

```
  }
};
```

Step 6: Route Validation Status (`IF_IsValidPayload`)

Add an **IF Node**:

- **Node Name**: `IF_IsValidPayload`
- **Condition**: `={{ $json.isValid }}` equals `{{ true }}`

Connect the **False Branch** to a **Code Node** named `Log_ValidationError`:

```
return {
  json: {
    event: "VALIDATION_FAILED",
    transactionId: $json.transactionId,
    errors: $json.validationErrors,
    timestamp: new Date().toISOString()
  }
};
```

Step 7: Call External Billing API with Retries (`HTTP_PostBillingCharge`)

Connect the **True Branch** of `IF_IsValidPayload` to an **HTTP Request Node**:

- **Node Name**: `HTTP_PostBillingCharge`
- **Method**: `POST`
- **URL**: `https://dummyjson.com/c/<your-id>` *(from Step 1 Option A, or your Postman Mock URL `https://{{mock-id}}.mock.pstmn.io/v2/charges`)*
- **Send Body**: `ON`
- **Body Content Type**: `JSON`
- **Specify Body**: `Using JSON`
- **JSON**:

```
{{
  {
    "transactionId": $json.sanitizedPayload.transactionId,
    "customerEmail": $json.sanitizedPayload.customerEmail,
    "amount": $json.sanitizedPayload.amount,
    "currency": $json.sanitizedPayload.currency,
    "description": $json.sanitizedPayload.description
  }
}}
```

- **Settings & Retry Policies** (Under Node Settings tab):
  - **Retry on Fail**: `ON`

All rights reserved. Tayana Academy • Confidential Document

- **Max Tries**: `3`
- **Wait Between Tries (ms)**: `5000`
- **Never Error**: `OFF` (allows fatal failures to escalate to Error Trigger)

Step 8: Build Success Audit Record (`Code_FormatSuccessRecord`)

Connect output of `HTTP_PostBillingCharge` to a **Code Node**:

- **Node Name**: `Code_FormatSuccessRecord`
- **Code**:

```
const response = $input.item.json;
return {
  json: {
    status: "PROCESSED",
    chargeId: response.chargeId || 'ch_generated',
    transactionId: response.transactionId || $node['Code_ValidateSchema'].json.sanitizedPayload.transactionId,
    amountCharged: response.amount,
    currency: response.currency,
    completedAt: new Date().toISOString()
  }
};
```

Step 9: Centralized Error Handler Configuration

1. Add an **Error Trigger Node**:
   - **Node Name**: `Error_Trigger`
   - Scope: Intercepts unhandled crashes across the workflow.
2. Connect to a **Code Node**:
   - **Node Name**: `Code_BuildIncidentAlert`
   - **Language**: `JavaScript`

```
const errorData = $input.first().json;
return {
  json: {
    alertType: "CRITICAL_PIPELINE_INCIDENT",
    workflowId: $workflow.id,
    workflowName: $workflow.name,
    failedNode: errorData.execution.error.node?.name || "Unknown Node",
    errorMessage: errorData.execution.error.message || "Unspecified execution failure",
    executionUrl: errorData.execution.url,
    timestamp: new Date().toISOString()
  }
};
```

All rights reserved. Tayana Academy • Confidential Document

5. Node-by-Node Settings Blueprint

Node ID	Node Name	Node Type	Critical Settings	Custom Expressions / Code
N1-01	`Webhook_Ingest`	Webhook	Method: POST, Path: `/v1/transactions`, Mode: Immediately, Code: 202	Immediate 202 JSON payload
N1-02	`Code_CheckIdempotency`	Code	Language: JavaScript, Run Once for Each Item	Static data cache check on `transactionId`
N1-03	`IF_IsNewTransaction`	IF	Mode: Basic, Condition: `isDuplicate == false`	`{{ $json.isDuplicate }} == false`
N1-04	`Log_BypassDuplicate`	Code	Format audit log for duplicate skip	`event: "TRANSACTION_BYPASSED"`
N1-05	`Code_ValidateSchema`	Code	Regex validation for email, positive number check, ISO currency	Full JavaScript schema validator
N1-06	`IF_IsValidPayload`	IF	Condition: `isValid == true`	`{{ $json.isValid }} == true`
N1-07	`Log_ValidationError`	Code	Format audit log for invalid payload	`event: "VALIDATION_FAILED"`
N1-08	`HTTP_PostBillingCharge`	HTTP Request	POST, Retries: 3, Delay: 5000ms	Dynamic JSON body from sanitized payload
N1-09	`Code_FormatSuccessRecord`	Code	Map charge ID, completed timestamp	Format audit object
N1-10	`Error_Trigger`	Error Trigger	Intercept unhandled exceptions	Global scope listener
N1-11	`Code_BuildIncidentObject`	Code	Extract failed node & stack trace	Format incident diagnostic payload

6. Verification Checkpoints & Testing Guide

Test 1: Valid Transaction Intake

Send a valid HTTP POST request via cURL:

```
curl -X POST https://n8n.tayanasolutions.com/webhook/v1/transactions \
  -H "Content-Type: application/json" \
  -d '{
    "transactionId": "TX-10001",
    "customerEmail": "sarah.connor@cyberdyne.io",
    "amount": 249.99,
    "currency": "USD",
    "description": "Enterprise Cloud Subscription"
  }'
```

- **Expected Result**: Immediate HTTP 202 Accepted. In n8n execution history, all nodes through `Code_FormatSuccessRecord` succeed with status `PROCESSED`.

Test 2: Idempotency Bypass Check

Send the identical request with `TX-10001` a second time:

All rights reserved. Tayana Academy • Confidential Document

- **Expected Result**: Webhook returns 202. `Code_CheckIdempotency` evaluates `isDuplicate: true`. Flow routes to `Log_BypassDuplicate` and halts; `HTTP_PostBillingCharge` is **NOT** executed.

Test 3: Schema Validation Rejection

Send an invalid payload (bad email, negative amount, lowercase currency):

```
curl -X POST https://n8n.tayanasolutions.com/webhook/v1/transactions \
  -H "Content-Type: application/json" \
  -d '{
    "transactionId": "TX-10002",
    "customerEmail": "not-an-email",
    "amount": -50,
    "currency": "us",
    "description": "Test Bad Order"
  }'
```

- **Expected Result**: Routes to `Log_ValidationError` detailing all 3 schema violations.

Test 4: External Server Outage & Retry Simulation

Send a transaction with `amount: 999` to trigger the mock server's HTTP 503 error:

- **Expected Result**: `HTTP_PostBillingCharge` retries 3 times with 5s delays. Upon final failure, `Error_Trigger` fires and logs the incident alert.

---

7. Production Risks & Mitigation Strategies

Risk	Impact	Mitigation
Memory Leak in Static Data Cache	High transaction volumes cause unbounded cache growth and n8n container OOM crash.	Implement array eviction limit (e.g. `staticData.processedTransactions.shift()` at 500 items) or swap to an external Redis instance.
Silent API Downtime	External payment provider outage leaves unbilled transactions without notification.	Global `Error_Trigger` captures exhausted retries and routes to automated Slack/Pager alerts.
Data Ingestion Format Drift	Upstream webhook caller changes payload schema (e.g. `amount` sent as string `"$249.99"`).	`Code_ValidateSchema` sanitizes types explicitly and rejects non-conforming entries.

All rights reserved. Tayana Academy • Confidential Document